

RenderReply — Live DM Inbox Engineering Spec

Comprehensive Frontend & Backend Implementation Guide for All Interactive Buttons and Components

Author: Rudra Teja	Platform: RenderReply SaaS	Version: 2.4 Production
Audience: Frontend & Backend Devs	Meta API: Graph API v20.0	Status: Official Implementation Spec

1. System Architecture & Meta Webhook Lifecycle

The **RenderReply Live DM Inbox** functions as a real-time conversational CRM bridging Instagram Direct, Reels comments, and Story mentions with automated keyword fulfillment and human agent intervention. Incoming events arrive via **Meta Webhooks** (HMAC SHA-256 validated), traverse an Ingestion Worker, and update the persistent state. The Frontend UI operates with optimistic updates and real-time state synchronization.

Layer	Component & Technology	Core Responsibility
Ingestion	Node.js / FastAPI Webhook Listener	Verifies Meta X-Hub-Signature-256; ingests DM/Comment payloads into Redis Queue.
Rules Engine	Regex & Trigger Pipeline Worker	Evaluates keywords (e.g., 'PRICING', 'JAVA'); triggers automated PDF delivery or store card DM.
Real-time Sync	WebSocket / Server-Sent Events	Pushes live thread updates, unread counters, and typing events to client browser in <50ms.
Outbound	Meta Graph API v20.0 Client	Dispatches human agent replies & product card templates via POST /me/messages.

2. Master Interface Map — Interactive Button Reference

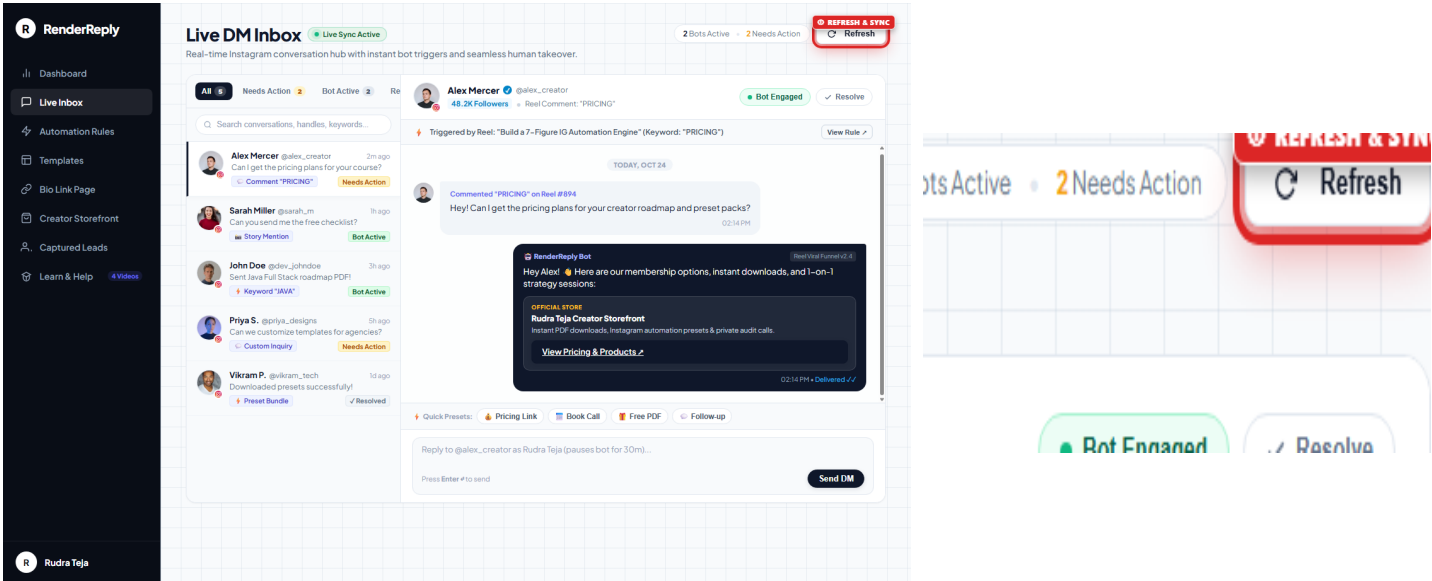
The screenshot below highlights all 10 core interactive buttons and components with red bounding markers and numbered pin badges:

3. Detailed Button-by-Button Technical Specifications

Each section outlines the exact visual highlighted screenshot, frontend event lifecycle, and backend API/database implementation.

Button 1: Refresh & Live Sync Button

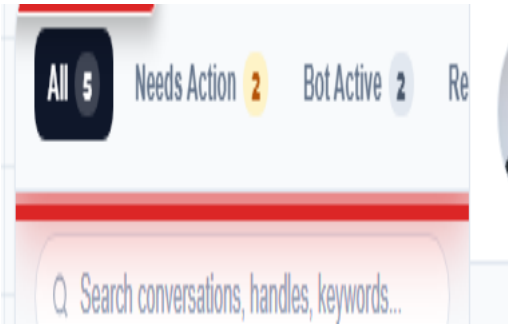
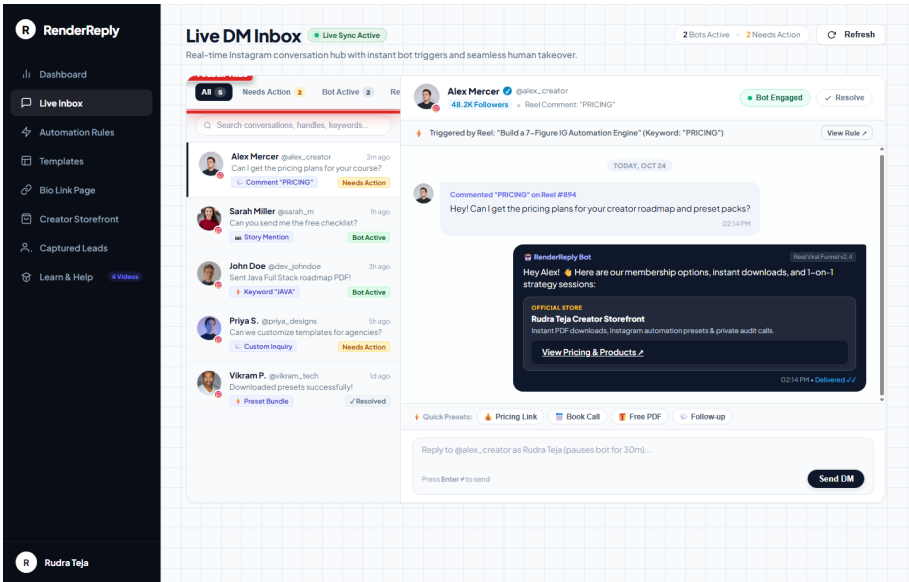
Summary: Triggers manual sync polling with Meta Instagram Graph API webhook queue to pull latest messages and refresh thread counts.



DOM Selector	#btn-inbox-refresh
Frontend Behavior	Listens for 'click' on #btn-inbox-refresh. Displays an optimistic toast ('Syncing real-time Instagram DMs & comments...'), updates status pills, and re-renders active thread messages.
Backend Endpoint	GET /api/inbox/sync
Backend & DB Operation	Calls GET /api/inbox/sync. Queries unread comments, story mentions, and DMs from Meta Graph API v20.0 /me/conversations. Updates PostgreSQL threads and messages tables with new webhook payloads. DB: UPDATE threads SET last_synced_at = NOW(), sync_status = 'synced'

Button 2: Folder Filter Navigation Tabs

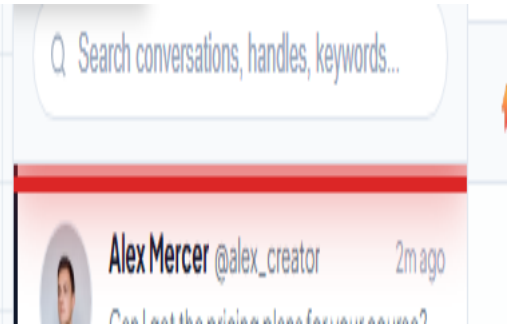
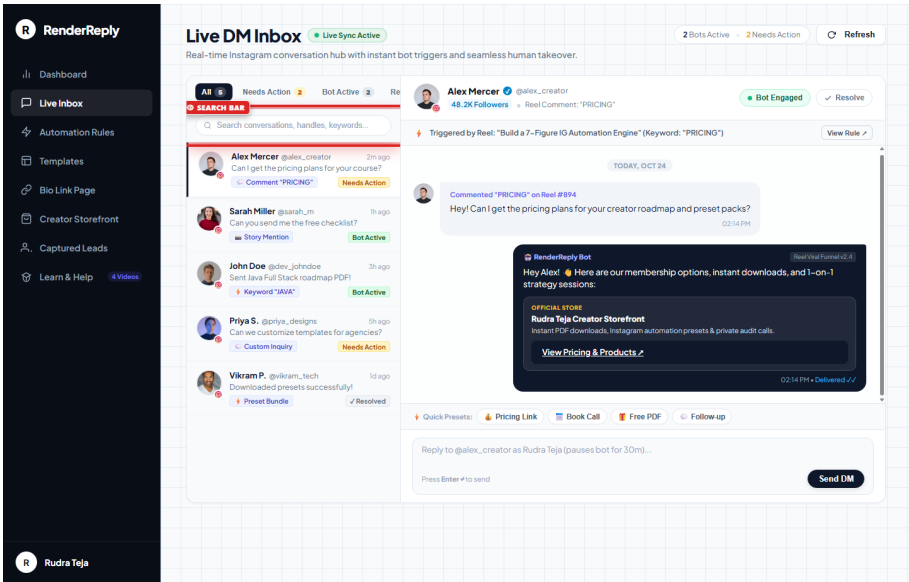
Summary: Stateful folder classification (All, Needs Action, Bot Active, Resolved) with real-time counter badges.



DOM Selector	.rr-clean-folder-tabs
Frontend Behavior	Listens for 'click' on .rr-folder-tab. Toggles .active class, reads data-filter attribute, and iterates through .rr-clean-thread-item elements to show/hide based on data-status.
Backend Endpoint	GET /api/inbox/threads?status={filter}
Backend & DB Operation	GET /api/inbox/threads?folder={all attention bot resolved}. Evaluates thread state machine: 'attention' (lead inquiries/unanswered), 'bot' (flow automated), 'resolved' (closed transactions). DB: SELECT * FROM threads WHERE creator_id = :uid AND status = :status ORDER BY updated_at DESC

Button 3: Real-Time Thread Search Input

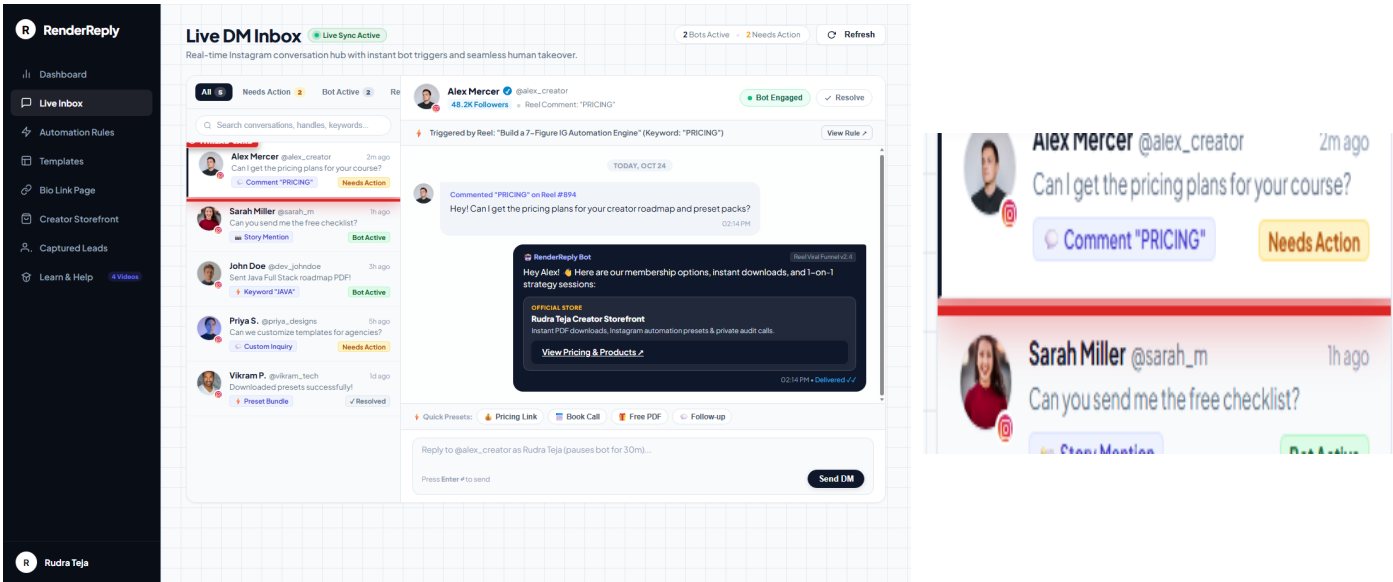
Summary: Instant client-side + debounced server-side fuzzy search across sender handles, names, snippets, and trigger keywords.



DOM Selector	.rr-clean-search-bar
Frontend Behavior	Listens for 'input' on #rr-inbox-search. Converts query to lowercase and performs instantaneous substring matching on sender name, Instagram handle, and message snippets.
Backend Endpoint	GET /api/inbox/search?q=:query
Backend & DB Operation	GET /api/inbox/search?q={query}. Performs full-text search across PostgreSQL (to_tsvector) or Elasticsearch index on sender profiles, trigger tags, and message histories. DB: SELECT * FROM threads WHERE search_vector @@ plainto_tsquery('english', :query)

Button 4: Conversation Thread Card Selection

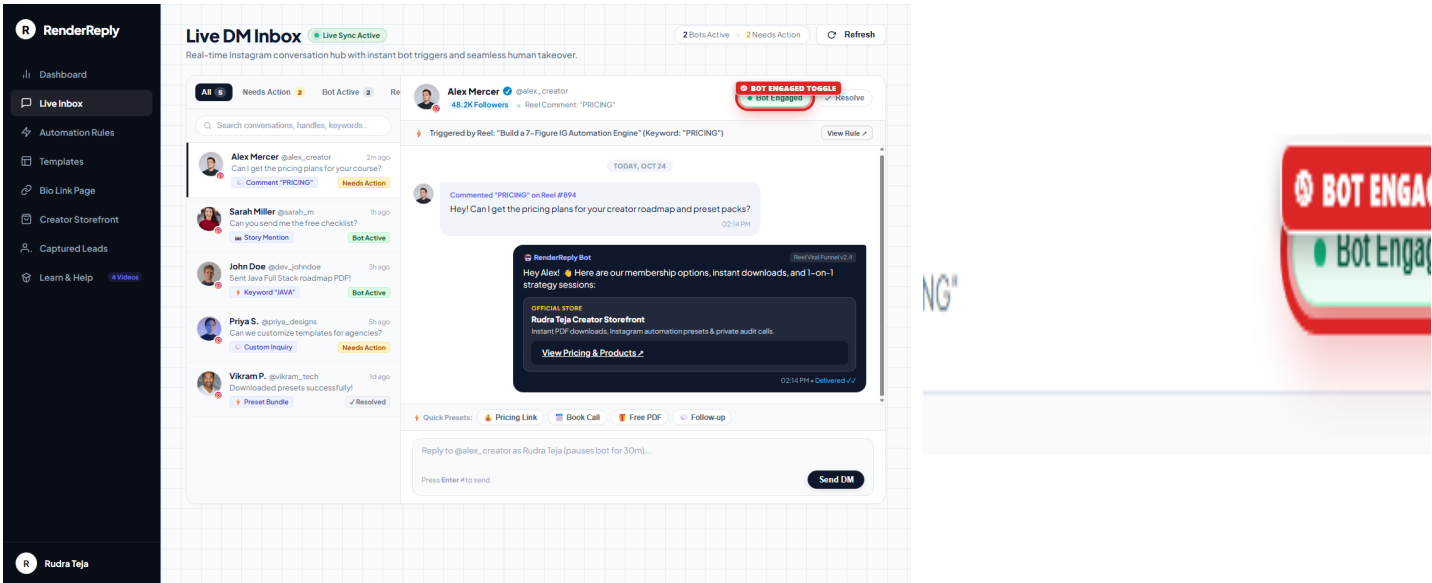
Summary: Displays contact avatar with Instagram Direct badge, follower tier, message snippet, trigger origin pill, and unread status.



DOM Selector	.rr-clean-thread-item.active
Frontend Behavior	Listens for 'click' on .rr-clean-thread-item. Invokes selectInboxThread(threadId), sets activeThreadId, populates chat header, trigger alert banner, and message feed.
Backend Endpoint	GET /api/inbox/threads/:thread_id/messages
Backend & DB Operation	GET /api/inbox/threads/:thread_id/messages. Fetches chronological conversation stream, marks message receipts as read via Meta Graph API POST /{message-id}/read. DB: UPDATE messages SET read_at = NOW() WHERE thread_id = :id AND sender = 'user' AND read_at IS NULL

Button 5: Bot Automation Pause/Resume Switch

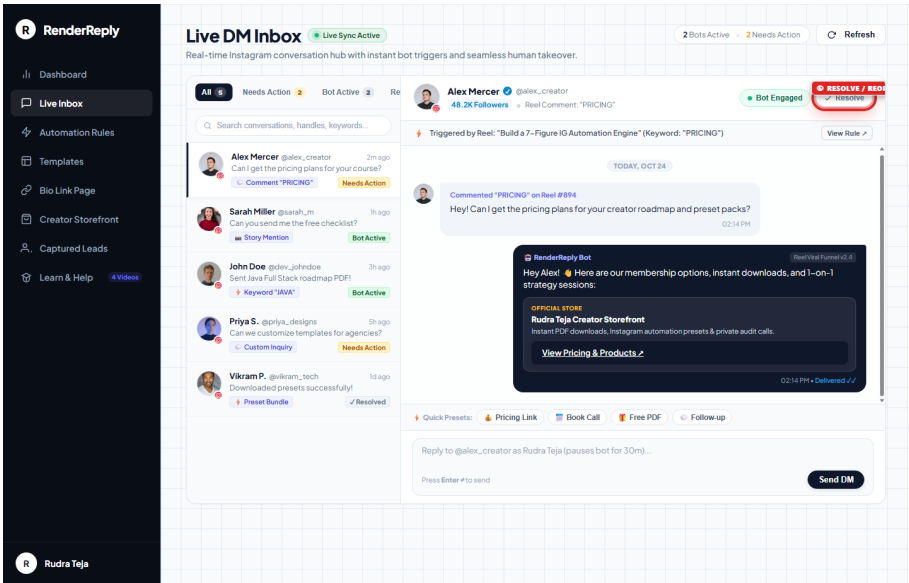
Summary: Dynamic AI/automation safety switch. Toggles between automated flow execution and 30-minute human takeover cooldown.



DOM Selector	#btn-toggle-bot
Frontend Behavior	Listens for 'click' on #btn-toggle-bot. Flips thread.botActive boolean, updates button styling (.paused vs active), dot indicator (.green vs .amber), and toast confirmation.
Backend Endpoint	POST /api/inbox/threads/:id/bot-toggle
Backend & DB Operation	POST /api/inbox/threads/:id/bot-toggle. Updates bot_active = {true false} in Redis/DB. If paused, sets Redis key 'bot_paused:{thread_id}' with TTL 1800s to suppress incoming bot triggers. DB: SET bot_paused:{thread_id} 1 EX 1800 (Redis) UPDATE threads SET bot_active = :val

Button 6: Resolve / Reopen Conversation Button

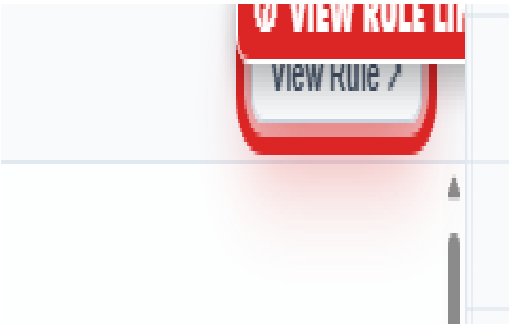
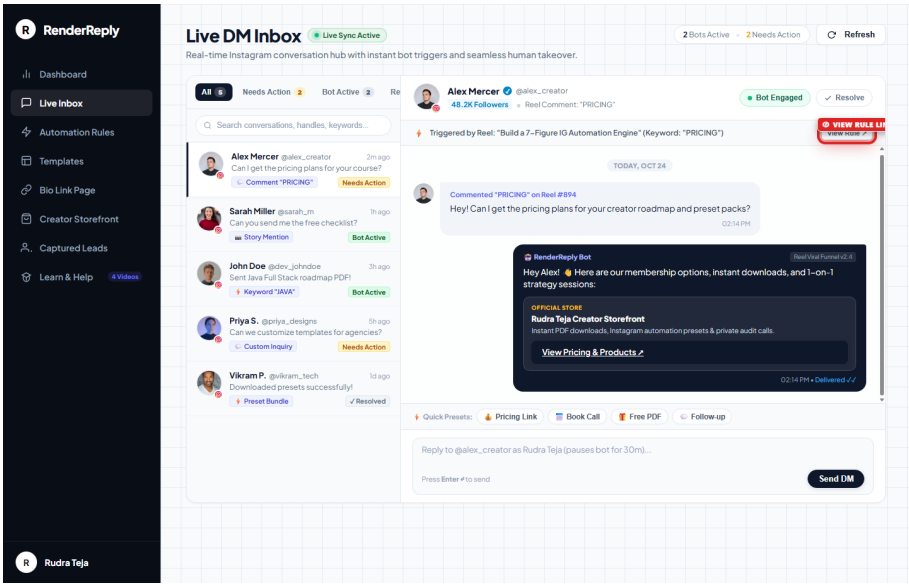
Summary: State machine transition action. Closes completed leads or reopens closed chats back into the Needs Action triage queue.



DOM Selector	#btn-resolve-chat
Frontend Behavior	Listens for 'click' on #btn-resolve-chat. Toggles thread.status between 'resolved' and 'attention', updates button label ('✓ Resolved (Reopen)'), updates thread badge, and recalculates folder counts.
Backend Endpoint	POST /api/inbox/threads/:id/resolve
Backend & DB Operation	POST /api/inbox/threads/:id/resolve. Updates thread status in DB, logs audit record in inbox_activity_log, and triggers webhook event 'thread.resolved' to creator CRM analytics. DB: UPDATE threads SET status = 'resolved', resolved_at = NOW() WHERE id = :id

Button 7: View Trigger Automation Rule Link

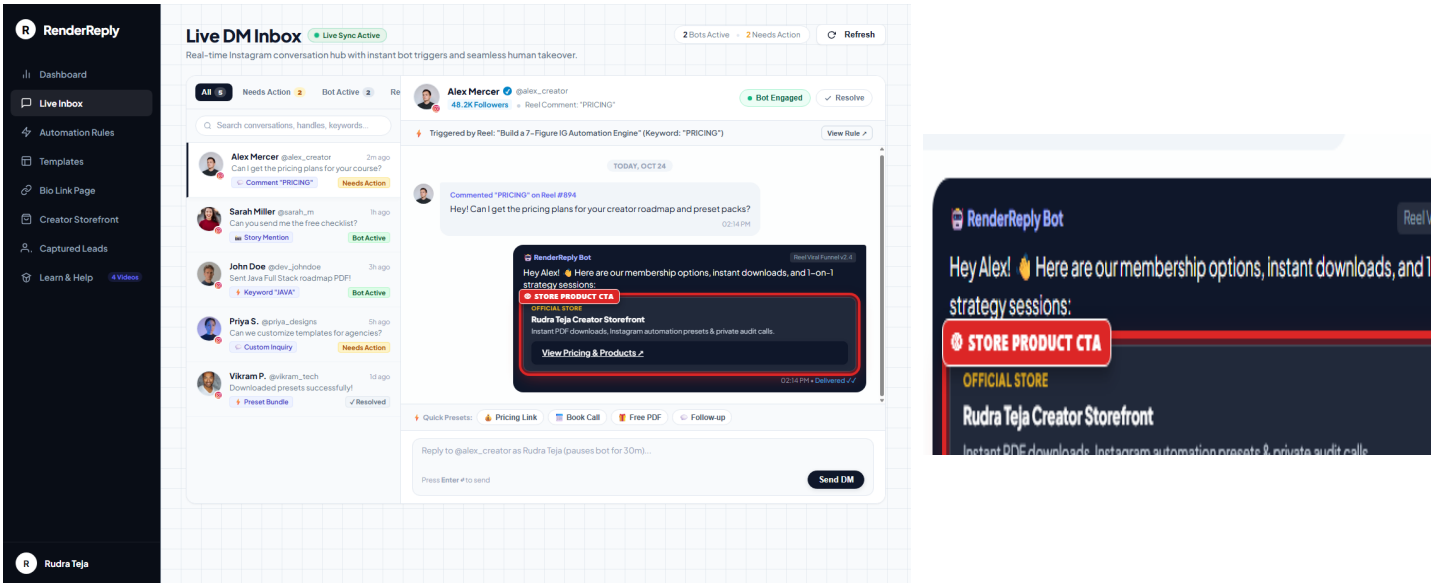
Summary: Direct deep link connecting the live conversation to its underlying automation rule in the RenderReply Automation Studio.



DOM Selector	#rr-tcb-view-rule
Frontend Behavior	Listens for 'click' on #rr-tcb-view-rule. Switches main application tab to 'automation-rules' and highlights the corresponding Reel keyword / Story mention automation flow.
Backend Endpoint	GET /api/automations/:rule_id
Backend & DB Operation	GET /api/automations/:rule_id. Retrieves the trigger definition, keyword matching regex, reply templates, product card attachments, and conversion tracking statistics. DB: SELECT * FROM automation_rules WHERE id = (SELECT rule_id FROM messages WHERE thread_id = :id LIMIT 1)

Button 8: In-DM Storefront Product Card & CTA

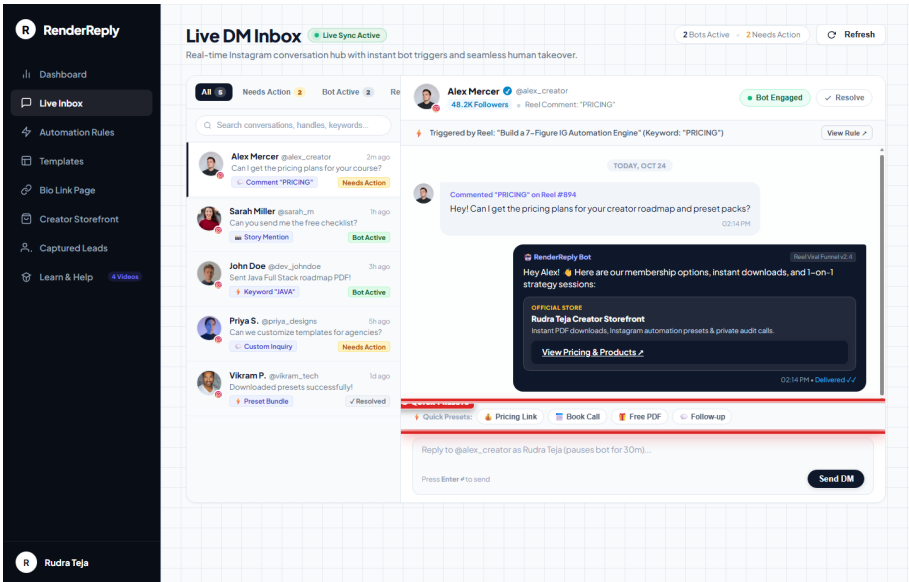
Summary: Rich interactive creator commerce card rendered directly inside Instagram Direct message stream with pricing & checkout CTA.



DOM Selector	.rr-clean-product-card
Frontend Behavior	Rendered via renderThreadChatFeed(). Displays official store badge, creator product headline, description, and external direct checkout link with click-tracking params.
Backend Endpoint	POST https://graph.facebook.com/v20.0/me/messages (Attachment Payload)
Backend & DB Operation	Dispatched via Meta Graph API Send API with Generic Template payload containing title, subtitle, image_url, and buttons[type='web_url', url='https://renderreply.com/store/rudrateja?ref=dm']. DB: INSERT INTO product_impressions (creator_id, thread_id, product_id, created_at) VALUES (...)

Button 9: Quick Action Preset Reply Chips

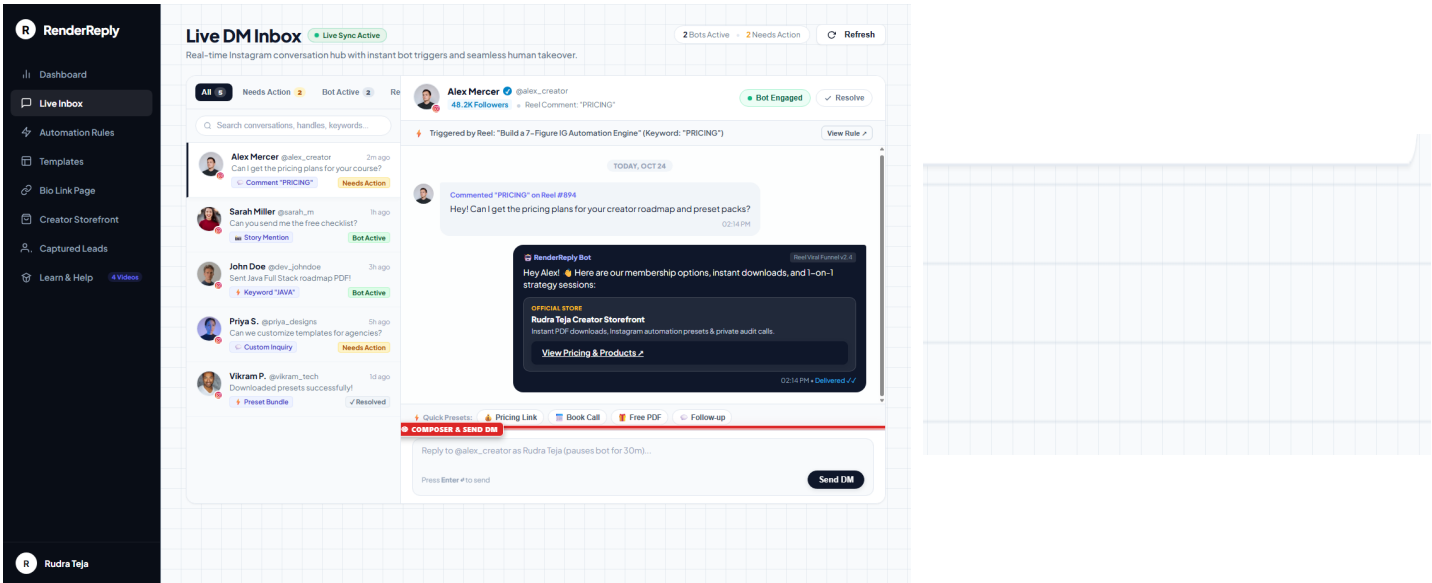
Summary: One-click macro reply shortcuts (Pricing Link, Book Call, Free PDF, Follow-up) for instantaneous creator responses.



DOM Selector	.rr-clean-preset-bar
Frontend Behavior	Listens for 'click' on .clean-preset-chip. Reads data-text attribute, injects formatted text directly into #chat-input-msg textarea, and automatically focuses the input cursor.
Backend Endpoint	GET /api/inbox/quick-replies
Backend & DB Operation	GET /api/inbox/quick-replies. Fetches creator's customized macro library. Allows creators to configure dynamic variables like {first_name}, {store_url}, and {calendar_link}. DB: SELECT * FROM quick_replies WHERE creator_id = :uid ORDER BY display_order ASC

Button 10: Direct DM Composer & Send Button

Summary: Unified messaging composer with keyboard Enter submission, human takeover badge, and official Meta Send API dispatch.



DOM Selector	.rr-clean-composer-box
Frontend Behavior	Listens for 'click' on #btn-send-chat and 'keydown' (Enter). Pushes message to thread.messages with optimistic UI append, auto-pauses bot, clears textarea, and triggers toast.
Backend Endpoint	POST /api/inbox/messages/send
Backend & DB Operation	POST /api/inbox/messages/send. Calls Meta Graph API POST /v20.0/me/messages with recipient.id and message.text. Returns message_id, saves to DB, and broadcasts delivery receipt. DB: INSERT INTO messages (thread_id, sender_type, body, meta_message_id, created_at) VALUES (...)

4. Backend Database Schema & REST Contracts

Production PostgreSQL database tables and JSON contract models required to support the Live DM Inbox:

PostgreSQL DDL Tables:

```
-- 1. THREADS CONVERSATION ENTITY
CREATE TABLE threads (
  id VARCHAR(64) PRIMARY KEY,
  creator_id VARCHAR(64) NOT NULL,
  contact_name VARCHAR(128) NOT NULL,
  contact_handle VARCHAR(128) NOT NULL,
  avatar_url TEXT,
  followers_count INT DEFAULT 0,
  status VARCHAR(32) DEFAULT 'attention', -- 'attention' | 'bot' | 'resolved'
  bot_active BOOLEAN DEFAULT TRUE,
  source VARCHAR(64), -- 'reel_comment' | 'story_mention' | 'dm_keyword'
  trigger_title TEXT,
  last_message_preview TEXT,
  last_activity_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- 2. MESSAGES ENTITY
CREATE TABLE messages (
  id VARCHAR(64) PRIMARY KEY,
  thread_id VARCHAR(64) REFERENCES threads(id) ON DELETE CASCADE,
  sender_type VARCHAR(16) NOT NULL, -- 'user' | 'bot' | 'human' | 'divider'
  body TEXT,
  context_tag VARCHAR(128),
  meta_flow_name VARCHAR(128),
  has_product_card BOOLEAN DEFAULT FALSE,
  product_id VARCHAR(64),
  delivered_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
  read_at TIMESTAMP WITH TIME ZONE
);
```

Meta Graph API v20.0 Outbound Store Card Payload:

```
// POST https://graph.facebook.com/v20.0/me/messages
{
  "recipient": { "id": "IG_SCOPED_USER_ID" },
  "messaging_type": "RESPONSE",
  "message": {
    "attachment": {
      "type": "template",
      "payload": {
        "template_type": "generic",
        "elements": [{
          "title": "Rudra Teja Creator Storefront",
          "subtitle": "Instant PDF downloads, Instagram automation presets & audit calls.",
          "buttons": [{
            "type": "web_url",
            "url": "https://renderreply.com/store/rudrateja?ref=dm_auto",
            "title": "View Pricing & Products"
          }]
        }]
      }
    }
  }
}
```